

TD7 - L'héritage en programmation objet

1) Problématique	1
2) Questions	1
3) Conclusion	7

1) Problématique

Nous cherchons à comprendre le fonctionnement du mécanisme d'héritage de la POO, pour cela nous allons répondre à plusieurs questions en rapport avec ce mécanisme.

2) Questions

Question 1 : Donner un exemple d'héritage.

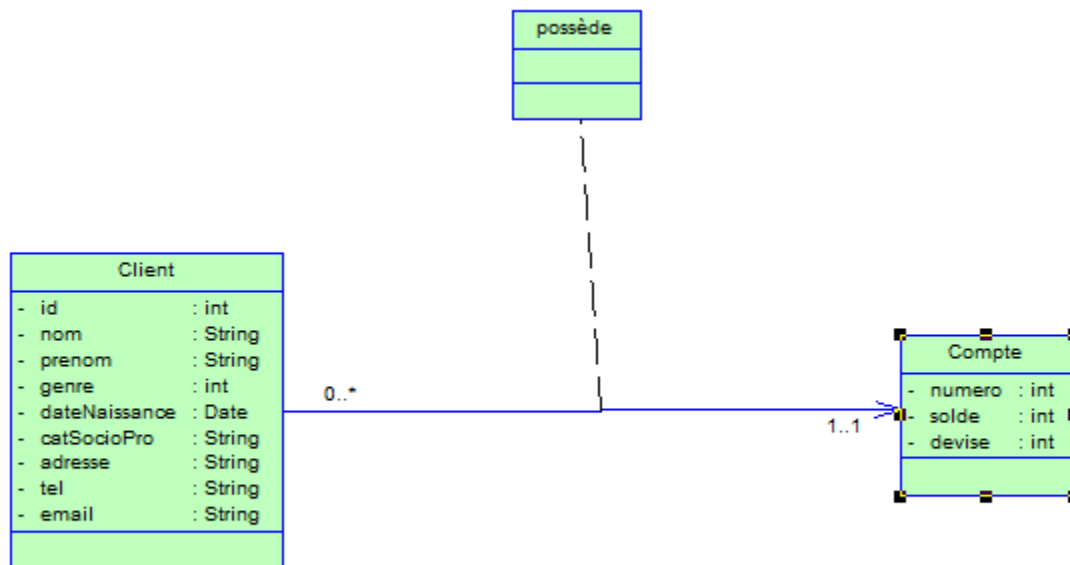
Imaginons une classe Animal, nous pouvons créer une classe enfant qui héritera de Animal et qui sera donc capable de réutiliser ou de remplacer les méthodes de la classe Animal. Voici l'exemple sous forme de code Java :

```
// Classe parent
class Animal {
    void parler() {
        System.out.println("Je fais un bruit");
    }
}

// Classe enfant
class Chien extends Animal {
    void parler() {
        System.out.println("Woof !");
    }
}

public class Main {
    public static void main(String[] args) {
        Chien monChien = new Chien();
        monChien.parler(); // Affiche "Woof !"
    }
}
```

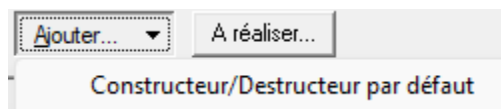
Question 2 : Faire le diagramme de classe de la banque.



Question 3 : Faire le constructeur, accesseurs et mutateur de la classe compte.

Pour faire cela c'est simple, il suffit de sélectionner l'attribut que l'on souhaite utiliser pour faire un setter et un getter approprié à celui-ci et cliquer sur .

Pour le constructeur, il suffit de cliquer sur **Ajouter** dans la section **Opérations** et de cliquer sur **Constructeur/Destructeur par défaut** :



Et lorsque cela est fait il suffit d'ajouter les paramètres au constructeur en double cliquant sur celui-ci, en allant dans paramètres et en remplissant les champs de façon adapté comme ici :

1	p_numero	pNumero	int	Entrée
2	p_solde	pSolde	int	Entrée
3	p_devis	pDevis	String	Entrée

Question 4 : Ecrire la méthode débiter et créditer

Pour créer les méthodes nous allons aller dans **Opérations** et ajouter aux deux nouvelles lignes les méthodes debiter et crediter :

8	debiter	debiter	void	public
9	crediter	crediter	void	public

Il ne faut pas oublier d'ajouter pour chacun de ces deux attributs le paramètre **montant** :

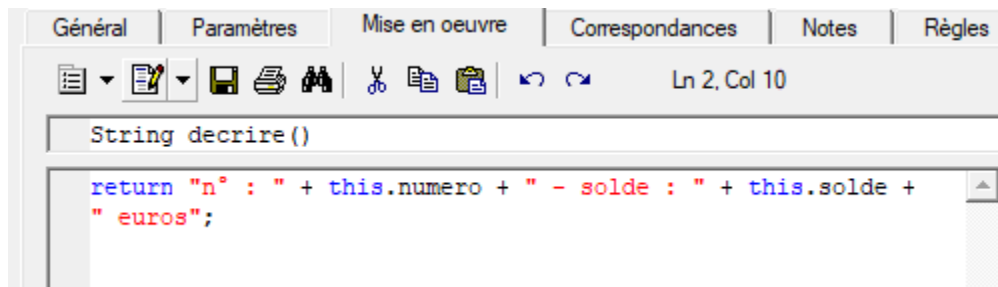
1	montant	montant	int	Entrée
---	---------	---------	-----	--------

Question 5 : Ecrire la méthode decrire de la classe compte qui retourne une chaîne de caractere avec la description d'un compte.

Pour faire cela on fait comme avant on ajoute la ligne decrire :

→	decire	decire	String	public
---	--------	--------	--------	--------

Puis nous allons aller dans l'onglet Mise en oeuvre après avoir double cliqué sur la ligne decire et ajouter le code de la méthode :

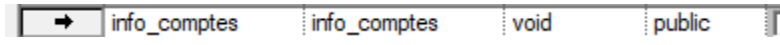


```
String decire()
{
    return "n° : " + this.numero + " - solde : " + this.solde + " euros";
}
```

(Il ne faut pas oublier d'appliquer pour chaque modification)

Question 6 :

Pour écrire la méthode **info_comptes** qui permettra d'afficher l'ensemble des comptes d'un adhérent, nous allons aller dans la table client, dans la section opération et nous allons ajouter la méthode de cette façon :

 , puis on applique.

Ensuite comme dans la question d'auparavant nous allons cliquer sur la section **Mise en oeuvre** présente dans les propriétés de la méthode et nous allons ajouter le code suivant :

```
System.out.println("Liste des comptes de M. " + this.nom + " " + this.prenom);  
  
// Nous avons au préalable nommé le rôle de la classe Compte en sesComptes  
for (Compte unCompte : this.sesComptes) {  
    System.out.println(unCompte.Decrire());  
}
```

Nous utilisons la méthode decrire pour l'ensemble des comptes et nous affichons la liste des comptes présentes dans l'arraylist caché "sesComptes" que l'on a nommé en changeant le nom de rôle de la classe Compte dans ses propriétés.

Question 7 :

Pour la question 7 j'ai dû générer les codes java avec PowerAMC puis j'ai mis le code de la classe compte dans compte sur bluej et celui de classe client dans client, puis j'ai créé un main où l'on utilise le constructeur de la classe client (que j'ai dû générer au passage dans le diagramme de classe). Le résultat est le suivant :

```
Affichage de la liste des comptes de Martin :  
n° : 13245411324 - solde : 2548200.0 €  
n° : 32544771289 - solde : 1000.0 €  
n° : 22544771289 - solde : -214.0 €
```

Question 8 :

Liste des attributs identiques entre le compte épargne et un compte "classique"	Liste des attributs différent entre le compte épargne et un compte "classique"
Numero Titulaire Solde Devise	Taux_interet

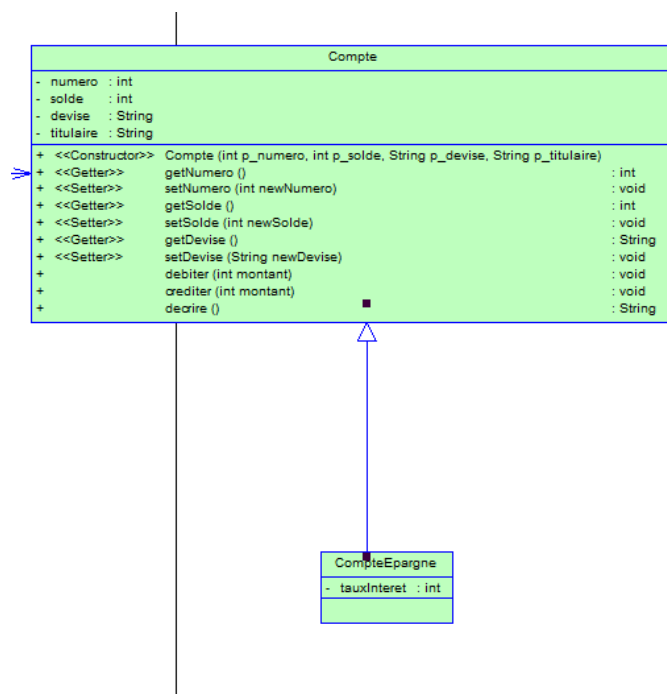
Question 9 :

Pour la question 9 nous allons ajouter la classe fille de Compte qui s'appellera CompteEpargne. Compte sera donc la classe mère.

Pour faire le système d'héritage nous allons utiliser l'outil **Généralisation**



et nous allons cliquer sur CompteEpargne et glisser jusqu'à Compte pour faire le lien parent-enfant.



Question 10 :

Nous allons ajouter titulaire, solde et numero aux attributs de la nouvelle classe puis nous allons générer les getters/setters de chaque attributs et enfin nous allons ajouter dans les paramètres du constructeur chaque attribut pour avoir le résultat suivant :

CompteEpargne		
-	tauxInteret : int	
-	numero : int	
-	solde : int	
-	titulaire : String	
+	<<Getter>> getTauxInteret ()	: int
+	<<Setter>> setTauxInteret (int newTauxInteret)	: void
+	<<Constructor>> CompteEpargne (int taux, int numero, int solde, String titulaire)	
+	<<Getter>> getNumero ()	: int
+	<<Setter>> setNumero (int newNumero)	: void
+	<<Getter>> getSolde ()	: int
+	<<Setter>> setSolde (int newSolde)	: void
+	<<Getter>> getTitulaire ()	: String
+	<<Setter>> setTitulaire (String newTitulaire)	: void

Question 11 :

Pour cette question nous allons créer une nouvelle méthode decrire dans laquelle nous allons rajouter le taux d'intérêt dans la section **Mise en oeuvre** avec ce code :

```
Propriétés de l'opération - decrire (decrire)
Général Paramètres Mise en oeuvre Correspondances Notes Règles Diagrammes associés Attributs étendus Dépendances Version
Ln 1, Col 115
void decrire ()
return "Compte Epargne n° : " + getNumero () + " - solde : " + getSolde () + " euros - taux = " + tauxInteret + "%";
```

Question 12 :

Nous allons créer la méthode `ajouter_interet` dans laquelle nous allons déclarer l'attribut `interets` qui s'appliquera au solde si celui-ci est positif :

```
void ajouter_interet()
{
    if (getSolde() > 0) {
        double interets = getSolde() * (tauxInteret / 100);
        setSolde(getSolde() + interets);
    }
}
```

Question 13 et 14 :

En ajoutant la classe de `CompteEpargne` dans BlueJ et en modifiant deux trois choses dans les autres classes nous pouvons faire en sorte d'afficher la liste des comptes du client avec le compte épargne en plus sur lequel on pourra voir le solde avant les intérêts et après les intérêts :

```
--- État des comptes avant intérêts ---
Client : Benjamin Martin
- Compte Classique n° : 13245411324 | Solde : 2548200.0 €
- Compte Epargne n° : 32544771289 | Solde : 1000.0 € (Taux : 5.0%)
- Compte Classique n° : 22544771289 | Solde : -214.0 €

--- État des comptes après intérêts ---
Client : Benjamin Martin
- Compte Classique n° : 13245411324 | Solde : 2548200.0 €
- Compte Epargne n° : 32544771289 | Solde : 1050.0 € (Taux : 5.0%)
- Compte Classique n° : 22544771289 | Solde : -214.0 €
```

3) Conclusion

En conclusion, ce TP m'a appris ce qu'était le polymorphisme et la façon dont on l'utilise en Java et sur PowerAMC, ça permet d'en apprendre plus sur la façon dont marchent les liens d'héritages des attributs de plusieurs classes différentes.